

그래프 표현학습 (Graph Representation Learning)

무작위 걷기와 정상분포 · Node2Vec: 걷기에 skip-gram · PageRank: 정상분포를
중요도로 · 확장: 단백질 · 금융 · 교통

Machine Learning 1 · 13주차

한국과학영재학교 (KSA)

Chapter 13

이번 주차 개요

- **13.1 Random Walk 기초** — 그래프 위 **무작위 걷기**가 마르코프 체인임을 정의하고, 카라테 클럽에서 걷기 말뚝치를 만들고, 노드에 머무는 장기 비율이 도(degree)에 비례함을 유도한다.
- **13.2 Node2Vec** — 걷기 경로 = “문장”, 노드 = “단어”로 word2vec의 **skip-gram**을 그대로 학습. $p \cdot q$ 두 계수가 탐색 스타일(BFS형 vs DFS형)을 어떻게 바꾸는지 작은 실험으로 본다.
- **13.3 PageRank** — “영원히 돌아다닌 뒤 각 노드에 머무는 확률은?”이라는 다른 질문. 댐핑 팩터 + 순간이동, 거듭제곱법 손계산, 그리고 두 가지 혼한 실수.
- **13.4 그래프 학습의 확장** — 두 아이디어를 실제 문제로: 단백질(AlphaFold → DeepFRI), 금융(AML World), 교통(STGCN).

그래프로 표현된 데이터를, 신경망이나 통계 모델이 다룰 수 있는 벡터로 어떻게 바꾸는가?

13.1 Random Walk 기초

이 장은 무엇을 묻는가 — AlphaFold가 준 관점

2021년 딥마인드의 **AlphaFold**(Jumper et al., 2021)는 아미노산 서열만 보고 단백질의 3차원 구조를 원자 단위 정확도로 예측해, 50년 가까운 생물학 난제를 사실상 해결했다.

- 핵심 관점 중 하나: 단백질을 **그래프**로 본다.
- 각 아미노산(잔기, residue) = **노드**, 공간적으로 가까운 잔기 쌍 사이의 관계 = **엣지**.
- “어떤 잔기가 어떤 잔기와 가까운가”를 알면 3차원 구조가 따라 나온다.

이 장의 질문은 AlphaFold 구조 자체를 유도하는 게 아니라 더 근본적인 것 — **그래프로 표현된 데이터를, 벡터로 어떻게 바꾸는가?**

핵심 실: word2vec 스킵그램의 그래프 재사용

14.2절의 스킵그램 원리

“문맥 안에서 **함께 나타나는** 단어는 **비슷한 벡터**를 갖는다”

- 이 원리를 그래프에 **그대로** 재사용한다.
- “문장”의 역할은 그래프 위의 **무작위 걷기**(random walk).
- 이 장은 14장(표현학습, PCA·word2vec)과 순환 구조: 14장에서 “함께 자주 나타나는 것은 가까운 벡터로 모인다”의 **텍스트** 버전을 배울 예정이고, 이 장은 그 적용 대상을 **텍스트가 아닌 그래프로** 바꾼 것을 먼저 보여준다.

뒤로 보면: “그래프 위에서 값을 반복해서 갱신한다”는 감각은 15장(잠재변수 생성모델)의 확률 모델과 ML2의 벨만 방정식 (반복 대입으로 고정점 수렴)으로 이어진다.

그래프 위 무작위 걷기 = 마르코프 체인

정의

노드 집합 $\{0, \dots, N-1\}$ 의 그래프 위에서 “지금 서 있는 노드”를 **상태**로 보고, 매 단계에서 **지금 노드의 이웃 중 하나를 골라 이동**하는 확률과정 = **마르코프 체인**(Markov chain).

- 상태 i 에서 j 로 건너가는 **전이확률** (transition probability):

$$P(i \rightarrow j) = \frac{1}{\deg(i)} \quad ((i, j) \in \mathcal{E}), \quad P(i \rightarrow j) = 0 \quad (\text{그 외})$$

- 이웃을 “동등한 확률로 골라” 이동 = **일괄적**(uniform) 무작위 걷기.

같은 체인, 다른 파라미터 — 다음 두 절의 예고

- **13.2 Node2Vec**: 이 전이확률에 p, q 두 계수를 붙여 **편향**(bias)시키는 변형.
- **13.3 PageRank**: 이 전이확률에 “**순간이동**”을 섞은 변형.

한 줄 정리

“같은 마르코프 체인 위에 파라미터만 바꾸는 것” — 같은 무작위 걷기에 서로 다른 질문을 던진다.

마르코프 체인의 핵심 성질: 무자기성

다음 상태를 결정하는 것은 “지금 어디에 있는지”뿐이다 — 그전에 어떤 경로를 거쳐 왔는지는 아니다.

$$P(x_{t+1} \mid x_t, x_{t-1}, \dots, x_0) = P(x_t \rightarrow x_{t+1})$$

- 이 한 줄이 무작위 걷기가 “단순한 반복”으로 분석될 수 있는 이유 — 그래서 **수십억 노드의 웹에도 거둬제공법 한 줄로** 돌릴 수 있다.
- 예: 카라테 클럽에서 node 16이 node 5를 거쳐 왔는지 node 6을 거쳐 왔는지 **기억하지 않고**, 오직 “지금 node 16에 서 있고 내 이웃은 5와 6이다”만 아는 것이 바로 이 성질.

상태 분포를 계속 곱하면 — 정상분포

- 전이확률 행렬 P 를 “각 노드에 서 있을 확률”인 상태 분포 π 에 계속 곱한다:

$$\pi_{t+1} = \pi_t P$$

- 분포는 결국 멈추는 **정상분포**(stationary distribution) π^* 에 도달한다:

$$\pi^* P = \pi^*$$

- “영원히 돌아다닌 뒤 각 노드에 머무는 비율”이 바로 이 π^* — 아래에서 도(degree)에 비례함을 직접 유도.

같은 무작위 걷기, 다른 질문

13.2 Node2Vec

- “이 걷기가 만들어내는 **노드의 순서(시퀀스)**는 어떤 구조를 담고 있는가?”
- 그 시퀀스를 “문장” 취급해 word2vec의 skip-gram을 학습 → **노드 임베딩**.

13.3 PageRank

- “이 걷기를 영원히 돌리면 각 노드에 머무는 확률 π^* 는 얼마인가?”
- 그 정상분포 자체를 노드의 “**중요도**”로 읽는다.

한 줄

같은 재료(그래프, 무작위 걷기)에서 하나는 **시퀀스**를 뽑아내고, 하나는 **정상분포**를 뽑아낸다 — 벡터와 스칼라, 두 가지 다른 표현.

왜 “무작위 걷기”가 “문장”을 대신하는 자연스러운 선택인가

- word2vec에서 “문장”이 가진 본질: 어떤 것들이 어떤 순서로 함께 나타나는지를 담는 것.
- 그래프 노드의 “함께 나타남” = “엣지로 연결되어, 서로를 따라다니면 만나는 것”.
- 어떤 노드에서 출발해 이웃을 무작위로 골라 반복 이동하는 **무작위 걷기**가 바로 이 “함께 나타남의 순서”를 만들어내는 **최소 장치**.
- 걷기 안에서 **인접한 노드는 서로 엣지로 연결되어** 있고, 어떤 노드 주위에 자주 등장하는 노드 = 그 노드의 구조적 “이웃”.

머무는 비율은 도(degree)에 비례한다

핵심 성질

무방향 그래프에서 일괄적 무작위 걷기를 무한히 계속하면, 노드 i 에 머무는 장기 비율은 그 노드의 **도**(degree, 연결된 엣지 수)에 비례:

$$\pi(i) = \frac{\deg(i)}{2|\mathcal{E}|}$$

($|\mathcal{E}|$ = 엣지 수, $2|\mathcal{E}|$ = 모든 노드의 도의 합)

- 즉 허브 노드는 걷기 말뚱치에 더 자주 등장 → skip-gram 학습에서 더 많이 업데이트.

허브는 더 많이 “나타나는 단어”

- 14.2절의 “**빈도 높은 단어의 임베딩 벡터 노름이 크다**” 는 현상과 **정확히 같은 메커니즘**.
- 카라테 클럽에서: node 33(관장) 도 17, node 0(감독) 도 16 — 두 파벌 리더가 구조적으로 가장 많이 “나타나는” 단어.
- 임베딩 학습이 자동으로 **구조적으로 중요한 노드에 더 많은 “학습 예산”**을 배정해준다고 볼 수 있다.

주석 — 이 절 후반과 13.3의 연결

“영원히 돌아다닌 뒤 각 노드에 머무는 비율”이라는 질문은 댐핑을 빼면 곧바로 **PageRank** 그 자체. 이 절의 후반부가 바로 이 질문에 정교한 답을 붙인다.

Zachary's Karate Club — 표준 테스트 데이터셋

- 1977년 인류학자 **웨인 재커리**(Wayne Zachary)가 한 가라테 동호회 **34명**의 친분 관계를 기록.
- 마침 이 동호회가 **감독(node 0)**과 **관장(node 33)** 두 파벌로 **실제 분열**.
- 34개 노드, **78개**의 친분 관계(edge)뿐인 작은 그래프.
- 그래프 임베딩이 “제대로 동작하는지” 확인하는 **표준 벤치마크**로 지금까지도 사용.

성공의 기준

라벨(파벌 소속) 없이 순수하게 그래프 구조만 보고 임베딩했는데도 두 파벌이 벡터 공간에서 자연스럽게 갈라지면 성공.

코드: 카라테 클럽 엣지 목록

```
KARATE_EDGES = [  
    (0,1),(0,2),(0,3),(0,4),(0,5),(0,6),(0,7),(0,8),(0,10),(0,11),  
    (0,12),(0,13),(0,17),(0,19),(0,21),(0,31),(1,2),(1,3),(1,7),(1,13),  
    (1,17),(1,19),(1,21),(1,30),(2,3),(2,7),(2,8),(2,9),(2,13),(2,27),  
    (2,28),(2,32),(3,7),(3,12),(3,13),(4,6),(4,10),(5,6),(5,10),(5,16),  
    (6,16),(8,30),(8,32),(8,33),(9,33),(13,33),(14,32),(14,33),(15,32),  
    (15,33),(18,32),(18,33),(19,33),(20,32),(20,33),(22,32),(22,33),  
    (23,25),(23,27),(23,29),(23,32),(23,33),(24,25),(24,27),(24,31),  
    (25,31),(26,29),(26,33),(27,33),(28,31),(28,33),(29,32),(29,33),  
    (30,32),(30,33),(31,32),(31,33),(32,33),  
]
```

무방향 그래프: 친분 관계 78개. node 0 = 감독, node 33 = 관장.

코드: 일괄적 무작위 걷기로 말뚱치 만들기

```
def random_walk(neighbors, start, length):
    walk = [start]
    for _ in range(length - 1):
        cur = walk[-1]
        if not neighbors[cur]:
            break
        walk.append(random.choice(neighbors[cur]))
    return walk

def build_walks(edges, n_nodes, walks_per_node=10, walk_length=8):
    neighbors = {i: [] for i in range(n_nodes)}
    for a, b in edges:
        neighbors[a].append(b)
        neighbors[b].append(a)
    walks = []
    for node in range(n_nodes):
        for _ in range(walks_per_node):
            walks.append([str(n) for n in random_walk(neighbors, node,
                                                         walk_length)])
    return walks
```

random_choice = 이웃을 동등한 확률로 고르는 일괄적 걷기. 각 걷기 = “문장”, 각 노드 번호 = “단어”.

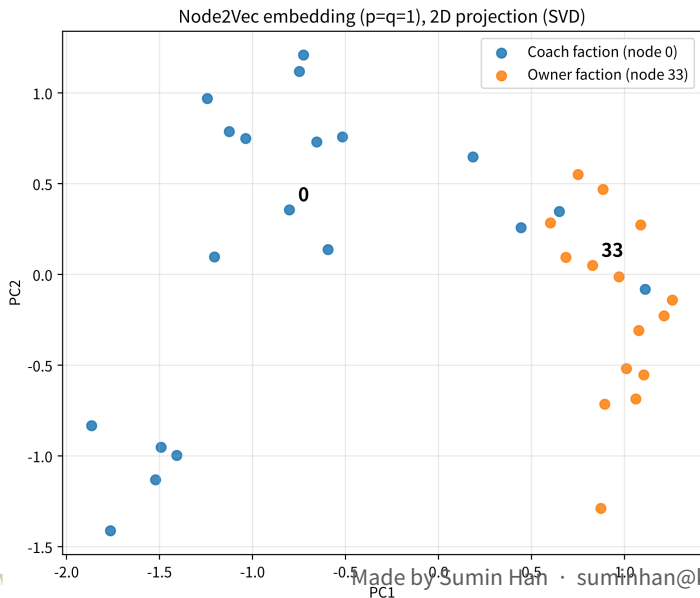
걷기 → 말뭉치 → skip-gram → 임베딩

- ① **걷기** — `build_walks` 로 34개 노드 × 노드당 10개 걷기(길이 8) 생성.
- ② **말뭉치** — 이어붙인다: `corpus = sum(walks, [])`.
- ③ **학습** — 14.2절의 `train_skipgram`에 그대로 넣으면 **34개 노드 각각의 임베딩 벡터**가 나온다.
- ④ **시각화** — 14.1절의 **PCA**로 2차원까지 압축해 그리면, 실제로 두 파벌이 공간적으로 갈라진다.

같은 아이디어의 두 얼굴

`word2vec`과 `Node2Vec`은 서로 다른 데이터(텍스트 vs. 그래프)에 적용됐을 뿐, “함께 자주 나타나는 것들은 비슷한 벡터를 갖도록 학습한다”는 같은 아이디어의 두 얼굴.

결과: 라벨 없이 본 임베딩이 분열 구조를 회복



$p=q=1$ (일괄적 걷기)의 걷기
 34×10 개(길이 8)로 Node2Vec
학습($d=8$ 차원)한 임베딩을 SVD로
2차원 투영. 파랑 = 감독파, 주황 =
관장파. 두 군집이 2차원 공간에서
상당히 선명하게 갈라진다.

해석: 리더는 “경계”가 아니라 군집 중심에

- 두 파벌이 한쪽/다른 쪽으로 갈라지고, 두 파벌 리더 (node 0, node 33)이 **각 군집의 중심 부근**에 있다.
- 두 리더의 임베딩이 자기 파벌 군집 중심에서 각각 **약 0.2, 0.3** 거리 — 나머지 멤버의 중앙값은 감독파 약 1.1, 관장파 약 0.4.
- “리더는 두 파벌의 **경계**에 있어야 할 것” 같은 직관과 달리, 실제로는 군집 **내부에 잘 박혀 있다**.
- 군집 경계 근처에 두 파벌을 잇는 “**다리**” 역할을 하는 멤버들이 섞여 있어도 **정상** — 그래프에서 실제로 두 클러스터 사이를 잇고 있다.

13.2 Node2Vec: 걷기에 skip-gram을 그대로

13.1을 한 줄로: 걷기 안의 인접 = “함께 나타남”

확립된 것

그래프 위의 **무작위 걷기**는 마르코프 체인이고, 그 걷기 안의 **인접** 노드들은 서로를 “함께 나타남”으로 잇는다.

- 이제 그 “함께 나타남의 순서”, 곧 **걷기 시퀀스**를 14.2절 word2vec의 **skip-gram** 학습에 그대로 넣으면 된다.

(Grover & Leskovec, 2016)

random walk로 생성한 경로들에 word2vec의 skip-gram을 그대로 학습시킨 것.

word2vec의 세 요소를 그래프 버전으로 치환

word2vec → Node2Vec

문장 → 한 노드에서 출발해 이웃을 무작위로 따라가며 만든 **걷기 경로**(노드 시퀀스)

단어 → 그 경로 위에 순서대로 나오는 **노드**

학습 → 14.2절의 train_skipgram을 **한 줄도 고치지 않고** 그대로 (토큰이 단어 대신 노드 번호일 뿐)

“비슷한 문맥에 나오는 단어는 비슷한 벡터를 갖는다” → “**비슷한 이웃 구조를 가진 노드는 비슷한 벡터를 갖게 된다**”는 문장으로 바뀐 것뿐.

학습이 끝나면: 은닉층 가중치 = 노드 임베딩

- 14.2절과 똑같이 은닉층 가중치 행이 각 노드의 임베딩 벡터.
- 14.2절에서 word2vec은 “작은 **신경망**을 학습시킨다”고 소개됐다 — one-hot 입력층 · 임베딩 차원 은닉층 · 어휘 크기 출력층, 이진 교차엔트로피 최소화.

핵심 재료가 신경망을 전제로 하지 않는다

“건기로 시퀀스를 만들고 그 위에서 함께-나타남을 학습한다”는 틀 자체는 **신경망 없이도 성립**. skip-gram을 쓰든, 더 단순한 “함께 나타난 횟수” 카운트 기반의 선형 분해 (14.2절의 LSI 같은)를 쓰든 — Node2Vec은 그 **더 일반적인 틀** 위에 14.2절의 skip-gram이라는 하나의 실용적 학습자를 골랐을 뿐.

Node2Vec의 진짜 기여: 두 계수 p 와 q

- 13.1절 코드의 일괄적 걷기 = 원본 논문의 **가장 단순한 특수한 경우** ($p=q=1$).
- 논문의 실제 기여: 걷기에 p (return)과 q (in-out) 두 계수를 붙여 “**이웃 탐색의 스타일**”을 조절.

걷기가 노드 x_t 에 도달했을 때(직전 x_{t-1}), 다음 노드 x_{t+1} 를 고르는 확률은 가중치 $w_{x_{t-1}}(x_{t+1})$ 에 비례:

- **돌아가기** — 방금 있던 노드로 복귀 ($x_{t+1} = x_{t-1}$): 가중치 $1/p$
- **인근 머물기** — x_{t-1} 의 이웃(이미 탐색한 로컬 영역 안): 가중치 $1/q$
- **새 영토로** — 나머지(아직 탐색 안 한 이웃): 가중치 1

계수를 움직이면 탐색 스타일이 기울어진다

- $p = q = 1$: 모든 가중치가 1 $\rightarrow$ 정확히 **일괄적 무작위 걷기**.
- $p < 1$ ($1/p > 1$): 직전 노드로의 **뒤로 감** (backtracking) 확률이 올라간다.
- $q < 1$ ($1/q > 1$): 방금 탐색한 **로컬 영역 안에 머물기** 확률이 올라간다.
- $q > 1$ ($1/q < 1$): 이미 본 영역을 다시 밟는 것이 **억제**되어 걷기가 **새 영토로 확장**하려 한다.

원본 논문의 두 표준 프리셋

설정	탐색 스타일	강조하는 구조
$(p=1, q=16)$ — “node2vec”	BFS형 (넓이 우선)	구조적 동치 (structural equivalence) — 역할의 유사성
$(p=16, q=1)$ — “deep walk”	DFS형 (깊이 우선)	동질성 (homophily) — 같은 커뮤니티 속한 것의 유사성

일괄적 무작위 걷기는 돌을 **섞어서** 담고, p, q 가 이 중 어느 쪽에 더 초점을 맞추게 하는 “**다이얼**” 인 셈.

동질성 vs 구조적 동치 — 서로 다른 두 유사도

동질성 (homophily)

- 같은 커뮤니티의 노드는 **동질**하다.
- 예: 카라테 클럽에서 **한 파벌의 멤버끼리** 서로 비슷.
- → DFS형 (16, 1) 가 강조.

구조적 동치 (structural equivalence)

- **서로 다른** 커뮤니티의 두 노드도 **역할이 같을** 수 있다.
- 예: 다른 부서 두 곳에서 각각 “외부와의 **유일한 연결고리**” 역할을 하는 두 멤버.
- → BFS형 (1, 16) 가 강조.

코드: 편향 무작위 걷기 (일괄적 걷기를 대체)

```
def biased_walk(nb, start, length, p, q, rng):
    """의Node2Vec 편향무작위걷기 . nb: 노드 -> 이웃목록 """
    walk = [start]
    prev = None
    for _ in range(length - 1):
        cur = walk[-1]
        weights = []
        for nxt in nb[cur]:
            if prev is None:
                weights.append(1.0)
            elif nxt == prev:
                weights.append(1.0 / p)      # 돌아가기
            elif nxt in nb[prev]:
                weights.append(1.0 / q)      # 로컬영역머무르기
            else:
                weights.append(1.0)          # 새영토로
        r = rng.random() * sum(weights)     # 가중치에비례해뽑기
        acc, chosen = 0.0, nb[cur][0]
        for nxt, w in zip(nb[cur], weights):
            acc += w
            if r <= acc:
                chosen = nxt
                break
        walk.append(chosen)
```

작은 실험: p 와 q 가 임베딩을 얼마나 바꾸는가

세 가지 설정으로 걷기를 **재생성**해서 동일하게 Node2Vec을 학습(시드 고정):

- **일괄적** (1, 1) — 일괄적 무작위 걷기
- **BFS형** (1, 10) — 새 영토 확장 강조
- **DFS형** (10, 1) — 로컬 영역 머물기 강조

공통 하이퍼파라미터: walks_per_node=10, walk_length=8, dim=8 (노트북).

결과

설정	코사인 실루엣*	node 2의 유사 노드 상위 3개
(1, 1) 일괄적	0.351	7 (0.73), 13 (0.72), 9 (0.72)
(1, 10) BFS형	0.342	9 (0.86), 28 (0.73), 7 (0.72)
(10, 1) DFS형	0.393	12 (0.83), 19 (0.70), 17 (0.65)

*실루엣

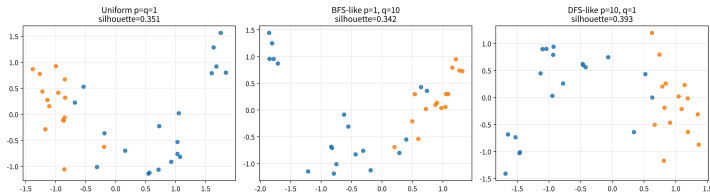
두 파벌 라벨을 “참 군집”으로 두고, 각 노드가 **자기 파벌 군집의 평균 임베딩**에 상대 파벌 군집의 평균보다 얼마나 가까운지 $-1 \sim +1$ 로 재는 군집 분리 품질 지수 (**1에 가까울수록 잘 분리**).

해석: 방향은 보인다, 크기는 작다

- 34개 노드라는 작은 스케일에서 세 설정의 차이는 **작고**, 원본 논문의 대규모 패턴까지 그대로 재현되지는 않는다 (데이터가 작아 노이즈가 큰 것은 자연스러움).
- 그래도 방향을 짚자: **DFS형** (10, 1)에서 node 2의 상위 3개 유사 노드(12, 19, 17)가 **모두 감독 파벌**에 속하는 **가장 깔끔한 파벌 분리**.
- 카라테 클럽에서 파벌 = 커뮤니티이므로, 여기서는 커뮤니티 분리 품질이 곧 “잘 됐다”의 지표.

세 설정을 한 그림에

Node2Vec embeddings by p and q (2D projection)



일괄적 (1, 1), BFS형 (1, 10), DFS형 (10, 1) 으로 각각 학습한 임베딩을 2차원 투영. 파랑 = 감독파, 주황 = 관장파. 세 설정 모두 두 파벌을 나누지만, **DFS형(실루엣 0.393)이 가장 선명하게** 갈라진다.

교훈은 이 실험이 아니라 다음 문장

p, q 는 “커뮤니티 구조를 원하느냐, 역할 구조를 원하느냐”에 맞게 바꾸고, 그 효과를 **downstream** 태스크(분류 · 링크 예측 등)로 **실제 검증**해야 한다.

- 이 실험 자체는 34개 노드의 작은 데모 — 큰 그래프에서 p, q 의 효과가 훨씬 크게 벌어진다(원본 논문).

자주 하는 실수: 무방향 엣지를 방향 그대로 넣고 돌리기

- 카라테 그래프는 **무방향**, PageRank는 **방향** 그래프 위에서 정의된다.
- 표준 기법: “친분 관계 1개”를 **양방향 링크 2개**로 바꿔주는 **대칭화** (13.3절의 directed_edges 줄).
- 이 대칭화를 **누락**하고 KARATE_EDGES를 그대로 넣으면, 결과는 **각 엣지가 어떤 방향으로 기록되었는가에 따라** — 즉 **임의적인 저장 순서에 따라** — 달라진다.

대칭화 누락의 결과: 감독이 “사라진다”

```
scores_dir = pagerank(KARATE_EDGES, 34) # 실수: 대칭화누락
# 상위개 5: 33 (0.0759), 32 (0.0280), 31 (0.0135),
#           16 (0.0125), 6 (0.0080)
```

- node 0(감독, 도 16의 진정한 허브)이 상위 5개에서 완전히 사라진다.
- 그래프 자체는 하나도 안 바뀌었는데, 기록 관례가 “중요도 순위”를 뒤틀어버린 것.

교훈: 도메인 지식과의 교차검증

발견하기 힘든 종류의 버그

대칭화는 **한 줄**이지만, 이 실수는 “**왜 감독이 순위에서 빠졌지?**”라는 도메인 지식의 교차검증이 없으면 발견하기 어렵다.

- 습관: 알고리즘 결과와 “알고 있는 사실”이 **충돌하면**, 먼저 **파이프라인의 편의 전제**(여기선 방향성 처리)부터 의심한다.

13.3 PageRank: 정상분포를 중요도로

같은 무작위 걷기, 다른 질문: PageRank

- 13.1: 걷기를 “노드가 이웃을 얼마나 자주 만나는지”를 담은 **말뭉치**로.
- 13.2: 그 말뭉치에 skip-gram을 학습 → **임베딩**.
- 이 절: 말뭉치가 아니라, 13.1의 마지막 주석 “영원히 돌아다닌 뒤 각 노드에 머무는 비율”에 정식으로 답 — 그 답, 즉 **무작위 걷기의 정상분포** 그 자체를 노드 하나당 스칼라 “중요도”로 쓰는 것이 **PageRank**.

같은 재료, 갈라진 결과물

같은 무작위 걷기를 13.2와 이 절이 모두 쓰지만, 묻는 질문이 달라서 나온 결과물이 임베딩 벡터와 스칼라 점수로 갈라진 것.

1998년: 래리 페이지와 세르게이 브린

- 1996-97년, 스탠퍼드 대학원생이던 **래리 페이지** (Larry Page)와 **세르게이 브린** (Sergey Brin).
- 당시 웹 검색엔진들은 “페이지에 **키워드가 있는가**”로만 순위를 매김 → 웹이 너무 커져 **쓸모없는 결과만 뒤섞여** 나온다는 문제를 직감.
- 그들의 답: **웹 자체를 데이터로 보는 것** — 수억 개 웹페이지 사이의 **링크 구조**는 그 자체로 **“웹의 투표 기록”**
- 다른 사이트의 주인들이 자발적으로 그 페이지를 링크한 것이 “이 페이지가 중요하다”는 **증거**.

PageRank의 핵심 아이디어와 정식화

- 이 그래프 위를 **무작위로 영원히 돌아다닌다면**, 각 노드에 머물러 있을 확률은 각각 얼마나 될까?
- 그 답 = 그 노드의 **“중요도”** (1998, Brin & Page).
- 많은 페이지가 링크하는 페이지일수록, 또 **중요한 페이지**가 링크하는 페이지일수록, **무작위 서퍼**(random surfer)가 더 자주 머문다.
- “누가 누구를 링크하는가”를 **무작위 서퍼의 이동 행렬**로 정식화 → 1998년 논문 “The Anatomy of a Large-Scale Hypertextual Web Search Engine”이 실용 알고리즘으로 완성.

(사명 “Google”은 10^{100} 을 뜻하는 “googol”에서 온 말장난 — 조직하려던 웹의 규모에 대한 선언.)

PageRank 방정식

노드 i 의 페이지랭크 $PR(i)$ 는 다음을 만족해야 한다 — 자신을 가리키는 모든 노드 j 로부터, 그 노드가 가진 점수를 그 노드의 바깥 링크 수만큼 나눠 받은 합:

$$PR(i) = \frac{1-d}{N} + d \sum_{j \rightarrow i} \frac{PR(j)}{\text{outdeg}(j)}$$

- N = 전체 노드 수.
- d (보통 **0.85**) = **댐핑 팩터**(damping factor): 확률 d 로 링크를 따라가고, 확률 $1 - d$ 로 **아무 노드로나 순간이동**(teleport).
- 순간이동 = **막다른 골목**에 갇히거나 **순환**에 갇히는 것을 막기 위한 장치.

재귀식이라 한 번에 못 풀고 — 거듭제곱법

- 이 식은 우변에 PR 자신이 다시 등장하는 재귀식이라 한 번에 풀 수 없다.
- 그래서 아무 값(모든 노드에 $1/N$)에서 시작해, 우변에 반복해서 대입하며 값이 더 이상 바뀌지 않을 때까지 되풀이하는 거듭제곱법(power iteration)으로 푼다.

형태가 수렴을 보장

“새 값 = teleport 항 + 옛 값들의 가중 평균”이라는 방정식의 형태 — 이 형태가 반복법의 수렴을 보장하는 구조이며, 바로 뒤에 온다.

코드: pagerank (거듭제곱법)

```
def pagerank(edges, n_nodes, d=0.85, iters=100):
    outgoing = {i: [] for i in range(n_nodes)} # directed: a -> b
    for a, b in edges:
        outgoing[a].append(b)
    outdeg = {i: max(1, len(outgoing[i])) for i in range(n_nodes)}
    incoming = {i: [] for i in range(n_nodes)} # who points to me?
    for a, b in edges:
        incoming[b].append(a)

    pr = {i: 1 / n_nodes for i in range(n_nodes)}
    for _ in range(iters):
        pr = {i: (1 - d) / n_nodes + d * sum(pr[j] / outdeg[j]
                                              for j in incoming[i])
              for i in range(n_nodes)}
    return pr
```

코드: 대칭화하고 카라테 클럽에 적용

```
# Karate 은 Club 무방향그래프이므로 ,  
# 친분관계하나를 양방향 링크로 취급한다  
directed_edges = KARATE_EDGES + [(b, a) for a, b in KARATE_EDGES]  
scores = pagerank(directed_edges, 34)  
print(sorted(scores.items(), key=lambda kv: -kv[1])[:3])  
# node 33, node 10 가장 높다 -- 실제 두 허브 """ 노드 관장(, 감독)과  
# 정확히 일치
```

(대칭화, $d=0.85$, 100회 반복)

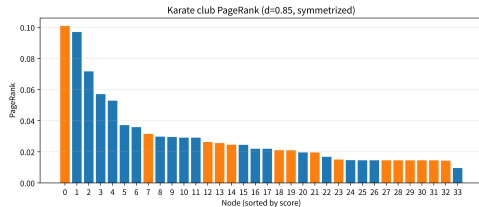
결과: 두 파벌 리더가 정확히 상위 2개

순위	노드	역할	PageRank
1	33	관장	0.1009
2	0	감독	0.0970
3	32	감독 파벌의 연결고리	0.0717

- 전체 점수의 합은 **정확히 1**.
- 최소값: teleport 하한 $\frac{1-d}{34} \approx 0.0044$ 보다도 조금 높은 **0.0096**(node 11, 도 1) — 하한은 “아무도 가리키지 않는 노드”가 받는 **이론적** 최저치일 뿐, 실제 그래프에서는 모든 노드가 누군가에게(직·간접으로) “가시게” 되니 그 위로 받쳐진다.
- 라벨을 하나도 안 본 “중요도 순위”가 **역사적 사실** (누가 어떤 파벌이었는지)과 거의 정확히 겹친다.

왜 node 33이 node 0보다 앞서는가

- node 33(도 17)이 node 0(도 16)보다 앞섬.
- “무작위 걷기가 더 자주 방문한다”는 도의 우위가 점수에 거의 그대로 드러난 모습.
- 상위권(33, 0, 32)은 모두 두 허브 노드와 그 연결고리.



34개 노드의 PageRank($d=0.85$, 대칭화)를 점수 높은 순으로 막대로. 파랑 = 감독파, 주황 = 관장파.
아래로 갈수록 점수가 teleport 하한에 수렴.

손으로 한 번: 노드 4개, 순환 하나, 외딴 끝 하나

$d = 0.5$ 인 방향 그래프로 최소 예제:

- $A \rightarrow B, B \rightarrow C, C \rightarrow A$ — **3-노드 순환**
- $D \rightarrow A$ — D는 A만 가리킨다 (들어오는 링크는 **없음**)

teleport 항

$$\frac{1-d}{N} = \frac{0.5}{4} = \mathbf{0.125}$$

1단계 — D는 들어오는 링크가 없다

- D를 가리키는 노드가 없으니 합 $\sum_{j \rightarrow D}$ 는 비어 있다:

$$PR(D) = \frac{1 - d}{N} = 0.125$$

읽기

D는 반복을 아무리 돌려도 0.125에 머문다 — “나를 가리키는 이가 없으니 **teleport**분밖에 받을 수 없다”는 뜻.

2단계 — 순환 속 3개 노드에 방정식

$$PR(A) = 0.125 + 0.5 \left(\frac{PR(C)}{1} + \frac{PR(D)}{1} \right)$$

$$PR(B) = 0.125 + 0.5 PR(A), \quad PR(C) = 0.125 + 0.5 PR(B)$$

(각 노드의 outdeg가 1이라 나누기는 일을 하지 않는다.)

3단계 — 대입으로 풀다

$PR(B), PR(C)$ 를 $PR(A)$ 식에 대입:

$$PR(A) = 0.125 + 0.5 \left(\underbrace{0.125 + 0.5(0.125 + 0.5 PR(A))}_{PR(C)} + \underbrace{0.125}_{PR(D)} \right) = 0.28125 + 0.125 PR(A)$$

$$PR(A) = \frac{0.28125}{0.875} = \frac{9}{28} \approx 0.321$$

```
PR(B) = 0.125 + 0.5 * 0.321 # ≈ 0.286
PR(C) = 0.125 + 0.5 * 0.286 # ≈ 0.268
# 계산: 0.321 + 0.286 + 0.268 + 0.125 = 1.000
```

이 작은 예제에서 읽어둘 것 두 가지

- 첫째. $A \cdot B \cdot C$ 는 모두 도 2(들어오는 1, 나가는 1)로 “겉모습은 똑같은”데, 점수는 **균등하지 않다** ($0.321 > 0.286 > 0.268$).
- “ $D \rightarrow A \rightarrow B \rightarrow C$ ” 흐름을 받는 A가 **D의 점수를 직접 받아 가장 크고, 순환을 돌수록 우위가 조금씩 소진된다.**

핵심

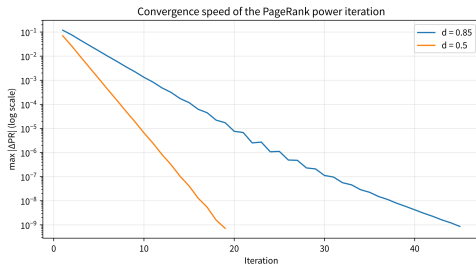
PageRank는 “엣지가 몇 개인가”가 아니라 “누가, 얼마나 중요한 위치에서 나를 가리키는가”의 문제.

거듭제곱법이 왜 수렴하는가 — 바나흐 고정점 정리

- 이 거듭제곱법의 수렴 논리는, **ML2 Chapter 4**에서 벨만 최적방정식의 해가 유일하게 존재함을 보일 때 쓸 **바나흐 고정점 정리**(Banach fixed point theorem)와 **정확히 같은 논리**.
- “어떤 변환을 값이 안 바뀔 때까지 반복해서 적용하면 결국 고정점(fixed point)에 도달한다”는 같은 수학:
- 여기서 = “그래프 위 확률분포”, 강화학습에서는 = “상태의 가치함수”.

수렴이 얼마나 빠른가 — 지수적으로

- 매 반복의 변화량(전체 노드 중 최대 $|\Delta PR|$)은 **지수적으로** 줄어든다.
- 카라테 클럽: $d=0.85$ 일 때 대략 **45회**로 10^{-9} 이내 수렴. $d=0.5$ 이면 **19회**.
- d 가 클수록 서퍼가 더 “장기 기억”을 하려 하므로 수렴이 **느려진다**.
- 작 중형 그래프: “**100회 반복**”이면 충분.
- 수십억 페이지 규모의 웹: 구글이 **분산 계산 · 희소화** 같은 별도 공학이 필요했던 이유.



매 반복의 최대 변화량 $\max |\Delta PR|$ 을 로그 스케어로. $d=0.85$ (파랑)가 $d=0.5$ (주황)보다 더 오래 걸리며, 두 곡선 모두 직선(= 지수적 감소)을 따라 10^{-9} 아래로.

실무 1: “수렴했는지” 확인 없이 반복 횟수만 고정하기

위의 코드는 단순화를 위해 `iters=100`을 고정했다 — 실제 사용에서는 **변화량이 기준 ϵ 아래로 떨어질 때까지** 돌리는 것이 정석:

```
def pagerank_converged(edges, n_nodes, d=0.85, tol=1e-9,
                      max_iters=1000):
    # ... 위( 와 pagerank 동일, 단 반복조건이 다름 )
    pr = {i: 1 / n_nodes for i in range(n_nodes)}
    for _ in range(max_iters):
        new = {i: (1 - d) / n_nodes + d * sum(pr[j] / outdeg[j]
                                                for j in incoming[i]) for i in range(n_nodes)}
        delta = max(abs(new[i] - pr[i]) for i in range(n_nodes))
        pr = new
        if delta < tol:
            break
    return pr
```

실무 2: dangling node (나가는 링크가 0인 노드)

- $\max(1, \cdot)$ 가 막는 것은 출도가 0인 전달자가 분모 0으로 만들지 않는 것이지, 그 노드 자신이 받는 teleport분 $\frac{1-d}{N}$ 을 없애는 것이 아니다.
- 그래서 “dangling 노드가 있으면 전체 PR의 합이 1이 안 된다”는 주장은 **잘못**. (2-노드 예제 $A \rightarrow B$ 만 있는 그래프에서도 B가 dangling이건 말건 합은 정확히 1.)
- 다만 출도가 0인 노드의 점수가 우변의 합에서 **어디로도 전달되지 않는다는 사실은 변함없다**.
- 카라테 그래프에는 그런 노드가 없어 합 = 1이 정확히 나왔지만, 웹처럼 “링크가 하나도 없는 죽은 페이지”가 흔한 그래프에서는 dangling 노드의 점수를 **모든 노드에 균등하게 재분배**해 주는 보정이 **표준**.

댐핑 팩터와 같은 계열의 공학 — “확률이 새는 구멍을 하나하나 막아주는” 장치들.

자주 하는 실수: “PageRank = 도(degree)”로 읽기

- “가리키는/가리켜지는 링크가 많을수록 점수가 크다”는 설명 때문에 PageRank를 “도 순위”로 단순화하는 경우가 많다.
- 무방향 그래프의 **대칭화**된 PageRank에서는 둘의 순위가 **매우 강하게** 상관난다(카라테 그래프에서도 상위권이 거의 도 순서와 일치).
- 그러나 이 둘은 **같은 것이 아니다** — $\sum_{j \rightarrow i} \frac{PR(j)}{\text{outdeg}(j)}$ 는 어떤 노드가 나를 가리키느냐를 가중.

본래 설계 목적

방향 그래프에서는 차가 확 벌어진다: “중요하지 않은 페이지 10개가 링크”보다 “**가장 중요한 페이지 1개가 링크**”가 더 높은 PageRank를 만드는 것. **인기(도)**와 **권위**(가리켜지는 쪽의 중요도)는 다른 개념이며, PageRank가 측정하는 쪽은 **후자**.

지금 어디에 쓰이는가

- 본업: 웹 검색 랭킹. 그러나 “방향 그래프 위 무작위 걷기의 정상분포 = 중요도”라는 틀 자체는 범용 도구.
- 학술 인용 네트워크(논문 · 저널 랭킹), 소셜 네트워크 영향력 분석.
- **개인화 PageRank**(Personalized PageRank): 무작위 걷기를 주어진 노드에서 다시 시작하는 변형 — 추천 시스템과 커뮤니티 감지에 사용.
- Node2Vec 쪽: 원본 논문에서 **100만 노드** 규모의 그래프 분류 · 링크 예측 · 시각화 시연. 이후 “그래프 → 노드 벡터 → 일반 ML 모델로 전달” 파이프라인의 첫 단계로 **약물 상호작용 예측, 단백질 기능 예측, NER** 등 다양한 그래프 문제에.

하나의 수학적 패턴

공유되는 패턴

PCA · word2vec · Node2Vec · PageRank는 얼핏 서로 다른 문제처럼 보이지만, 전부 “무언가를 어떤 변환에 반복해서 통과시키면 특별한 지점(고유벡터, 임베딩, 정상분포)에 수렴한다”는 하나의 수학적 패턴을 공유한다.

지도 vs 비지도

지도학습이 “정답을 맞추는 법”을 배운다면, 비지도학습은 “데이터가 스스로 어떤 모양을 하고 있는지”를 배운다 — 둘은 서로 다른 질문에 답한다.

확인 문제: 2-노드 그래프

2-노드 방향 그래프 $A \leftrightarrow B$ ($A \rightarrow B, B \rightarrow A$)에 $d = 0.5, N = 2$ 로 PageRank를 손으로 계산.

- ① **단계 1** — 방정식 두 개: $PR(A) = 0.25 + 0.5 PR(B), PR(B) = 0.25 + 0.5 PR(A)$
- ② **단계 2** — 대칭성: $PR(A) = PR(B)$ 로 놓으면 $PR(A) = 0.25 + 0.5 PR(A)$, 즉 $PR(A) = PR(B) = 0.5 - d$ 의 값과 무관하게 각 0.5. (코드로 확인)
- ③ **단계 3 (핵심)** — $B \rightarrow A$ 엣지를 지우면 (B는 **dangling node**가 됨): $PR(A) = 0.25, PR(B) = 0.25 + 0.5 \times 0.25 = 0.375$ — 합이 $0.25 + 0.375 = \mathbf{0.625}$ 로, **1이 아니다. 왜 그런지, 그리고 본문의 어떤 공학 보정을 추가하면 합이 1로 돌아올지 이름으로 답하라.** (힌트: “자주 하는 실수” 섹션의 두 번째 세부 사항)

자주 묻는 질문 (1/2)

- **Q. PageRank도 임베딩인가요?** — 아니요. Node2Vec은 각 노드를 **벡터 공간의 한 점**(임베딩)으로, PageRank는 각 노드에 **스칼라 값**(중요도 점수) 하나만. “구조적으로 얼마나 비슷한가”(Node2Vec) vs “전체에서 얼마나 중요한가”(PageRank).
- **Q. 댐핑 팩터 d 가 없으면?** — $d=1$ 이면 순간이동이 전혀 없어 막다른 골목에서 서퍼가 **영원히 갇히거나**, 그래프가 여러 조각으로 나뉘어 있으면 **시작 조각에 따라 결과가 달라짐**. $d<1$ 의 순간이동 확률은 확률분포가 조각 사이를 계속 넘나들게 만들어 **유일한 해로 수렴**을 보장.
- **Q. 걷기 개수·길이는?** — 엄격한 공식은 없지만 원본 논문의 **노드당 10개 걷기, 길이 40, 차원 128**을 출발점으로 데이터 규모에 맞춰 줄인다(34개 노드에서는 10개 $\times$ 길이 8 $\cdot$ 차원 8이면 충분). 너무 짧으면 “문맥”이 바로 옆 이웃에 갇히고, 너무 길면 “출발 지점”을 잊어 문맥이 노이즈가 된다.

자주 묻는 질문 (2/2)

- Q. 무방향 그래프에 써도 되나요? — 된다. 표준 기법은 무방향 엣지 1개를 **방향 링크 2개**로 바꿔주는 것. 이렇게 하면 “도 순서에 가깝되, 누가 나를 가리키느냐를 가중한” 순서가 되며, 일반적으로 도 순위와 동일하지는 않다.
- Q. p, q 는 그래프마다 튜닝해야 하나요? — 보편적 최적값은 없다. 두 프리셋 $(1, 16), (16, 1)$ 도 “역할 구조 vs 커뮤니티 구조”에 맞춘 **출발점**에 불과 — 몇 가지 설정을 돌려보고 **downstream 태스크** 성능이나 시각화에서 기대된 구조가 보이냐로 검증한다.

13.4 그래프 학습의 확장

두 아이디어 → 세 실전 방향

- 이 장에서 배운 두 아이디어: **무작위 걷기 기반 그래프 임베딩**(Node2Vec)과 **정상분포**(PageRank).
- 오프너의 AlphaFold 이상으로 **훨씬 넓은 문제**에 적용된다.

세 가지 실전 방향 (지도식 수업)

- ① **(a) 단백질**: 구조를 알면, 그 구조로 기능을 분류
- ② **(b) 금융**: 거래 그래프에서 자금세탁 탐지 (IBM AML World)
- ③ **(c) 교통**: 도로망 + 시계열의 스페이오템포럴 그래프 교통 예측(STGCN)

(a) 단백질: 구조를 알면, 그 구조로 기능을 분류한다

- 오픈너의 **AlphaFold**는 잔기 쌍 사이의 **거리 · 각도 관계**를 그래프로 표현해 3차원 구조를 예측.
- 구조를 알고 나면 자연스러운 다음 질문: “이 구조는 어떤 기능을 하는가?”
- **DeepFRI**는 단백질 3차원 구조에서 얻은 **잔기 접촉 그래프**(contact graph)를 **그래프 신경망**(GCN)에 넣어, 그 단백질이 어떤 **생물학적 기능**을 갖는지 분류.

자연스러운 파이프라인

구조 예측(AlphaFold) → 구조 기반 기능 분류(DeepFRI)

이 장의 도구가 어디에 쓰이는가

- 구조를 3차원 좌표로 돌려받는 순간, “공간적으로 가까운 잔기 쌍”을 엣지로 묶으면 **접촉 그래프**가 만들어진다.
- 카라테 클럽의 친분 엣지를 “친하다”로 바꾼 것 같은 — **도메인 지식이 만든 그래프**.
- 그래프 신경망 계열은 이 그래프에 13장 전체를 관통한 “**이웃과 값을 합치면서 반복한다**”는 연산 — **그래프 컨볼루션**(graph convolution) — 을 적용.
- 각 잔기(노드)의 벡터를 **이웃 잔기의 정보와 섞어 몇 라운드 갱신**하면, “이 잔기는 구조적으로 어떤 위치에 있는가”가 인접성만큼 벡터에 스며든다.
- DeepFRI는 이렇게 쌓인 노드 표현을 받아 **수십만 개의 기능 라벨**(GO term) 중 해당 단백질을 설명하는 것을 골라내는 **분류 문제**로 만든다.

왜 서열을 바로 분류하지 않고 구조 그래프를 거치는가

“왜?”를 한 번

동일한 기능을 가진 단백질은 **서열이 많이 달라도 3차원 형태(구조)는 비슷하게 유지되는 경우가 흔하다.**

- 구조 그래프는 “**서열과 무관한 기능의 공통 요인**”을 더 직접적으로 담는 표현.
- 같은 구조에 다른 서열의 단백질(**유사 동종단백질**) 기능을 추론할 때 **서열 기반 모델보다 유리.**
- AlphaFold의 구조 예측이 생물학 난제를 풀기만 한 게 아니라, **그래프 기반 추론의 입력을 대량으로 공급하는 인프라**가 된 이유.

노트북 Part 1: 파이프라인의 뼈대

- 작은 합성 접촉 그래프(20개 노드)를 만들고.
- “접촉 수(도)를 특징으로 한” 매우 간단한 분류기를 돌린다.

주의: 이것은 DeepFRI의 GCN을 직접 학습하는 것이 아니다

“그래프 구조 특징 → 기능 분류” 파이프라인의 뼈대를 느끼는 데모.

(b) 금융: 거래 그래프에서 자금세탁 탐지

- **IBM AML World**: 계좌를 노드, 거래를 엣지로 하는 대규모 합성 거래 그래프 데이터셋.
- 자금세탁은 흔히 “여러 계좌를 거쳐 돈의 출처를 세탁”하는 패턴(**레이어링**)으로 나타나는데 —
- 개별 거래 하나만 보서는 안 보이고, 그래프 구조 전체(누가 누구와 얼마나 자주, 어떤 패턴으로 거래하는가)를 봐야 드러난다.
- Node2Vec 같은 그래프 임베딩이나 그래프 신경망으로 “의심스러운 부분그래프”를 탐지하는 방향의 대표적 실전 응용.

왜 합성 데이터셋인가 + 레이어링의 특징

합성인 것의 의미

“**답이 알려진 거래 그래프**”를 만들어서 탐지 모델의 성능을 **측정**할 수 있게 하는 것 — 실제 금융 데이터는 **프라이버시** 때문에 쉽게 공개될 수 없으니, **구조와 노이즈 수준만 조절**해서 “탐지 가능한 패턴”을 심어놓은 합성 데이터셋이 연구용으로 나온 것.

- 레이어링 패턴의 특징: 불법 자금의 흐름이 **1-2홉이 아니라 3-4홉 이상**으로 뻗어 있다.
- “계좌 A가 B로, B가 C로, C가 해외로” 이동하는 동안 **각 구간은 정상 거래와 구분하기 어렵다.**

이 장의 두 도구를 거래 그래프에

노드 임베딩 (Node2Vec 계열)

- 계좌를 “거래 네트워크에서 **어떤 위치에 있는가**”로 벡터화.
- 동일한 패턴으로 세탁에 쓰인 계좌들이 임베딩 공간에서 **같은 군집**에 모이면, 그 군집 자체가 “의심 패턴”의 **시그니처**.

중요도 점수 (PageRank 계열)

- “다른 계좌들이 **가장 많이** 거쳐가는 **통로**”에 높은 점수.
- 자금세탁의 **중계 계좌**(여러 출처를 여러 목적지로 합치고 흩어주는 허브) = 정확히 그런 **구조적 허브**.

두 신호를 결합하면

“무슨 특징인지 아직 이름이 붙지 않았지만” **구조적으로 수상한 계좌**를 선별 — 규칙 기반 탐지(“거래액 > 1억이면 신고”)가 놓치는 **패턴 전체가 정상인 개별 구간**까지 커버.

노트북 Part 2: 합성 거래 그래프

- 작은 합성 거래 그래프(20개 계좌, 정상 클러스터 + 의도적으로 심은 레이어링 경로)를 만들고.
- PageRank로 “중계 허브”를 찾아보며.
- 1홉 특징만으로는 왜 발견 못 하는지 보여준다.

(c) 교통: 도로망 위의 시계열 — spatio-temporal 그래프

- 지금까지: 그래프 구조 자체가 고정 (카라테 클럽의 친분 관계는 안 바뀜).
- 교통 예측처럼 그래프 위에 시계열이 얹히는 문제도 있다:
- 도로망(그래프)은 고정이지만, 각 도로 구간의 교통량은 시간에 따라 변한다.
- **STGCN**(Spatio-Temporal Graph Convolutional Network): 그래프 컨볼루션(공간 방향: “이웃 도로의 정보를 합친다”)와 시계열 모델(시간 방향: “과거 몇 스텝을 본다”)을 결합.

주의

비슷한 계열의 **DCRNN**도 있는데, 그 논문의 “**diffusion convolution**”은 15장 생성모델의 diffusion과는 다른 개념 — 여기서는 그래프 위의 확산 과정(무작위 걷기와 비슷한 발상)을 가리킨다.

왜 두 방향이 모두 필요한가

공간 방향만 — “이웃의 교통량을 합친다”

- **시간에 따른 변화**를 담을 수 없다: 출근 · 퇴근 시간대의 **파도**, 신호에 따른 **국소 정체**.

도로망은 그 **자신이 그래프**다 — 교차로가 노드이고 도로가 엣지.

시간 방향만 — “이 도로의 과거 교통량만 본다”

- **다른 도로의 영향**을 놓친다: 인접 구간의 정체가 이 구간으로 **전파**.

STGCN: 공간 $\times$ 시간을 순환적으로 결합

- ① 한 단계에서 (1) **공간 방향**으로 이웃 도로의 정보를 합치고,
- ② (2) **시간 방향**으로 과거 몇 단계의 **자기 자신**을 본다.
- 이것을 반복하면 “**인접 도로의 과거 흐름이 이 도로의 미래 흐름에 어떤 영향을 미치는가**”를 학습.
- 이 “**공간 $\times$ 시간 순환**” 구조는 13장 전체의 “**반복해서 값을 갱신한다**”는 패턴의 자연스러운 확장 (**노드 반복 $\rightarrow$ 노드 + 시간 반복**).

노트북 Part 3: 합성 도로망

- 작은 **합성 도로망**(4×4 격자, 교차로 = 노드)에서 인공적인 “**출근 파도**” 시계열을 만들고.
- 두 모델을 비교:
- (a) **1홉 이웃 평균만 보는 모델**
- (b) **3홉 이웃 평균 + 과거 3단계 자기 평균을 보는 모델**
- **결과: 이웃 범위를 넓히고 과거를 함께 보면 오차가 줄어든다.**

세 사례를 한 장으로

사례	그래프(자료)	이 장의 도구	질문
(a) 단백질	잔기 접촉 그래프	그래프 컨볼루션(GNN)	“이 구조는 어떤 기능을 하는가?”
(b) 금융	거래 그래프	노드 임베딩 + 중요도 점수	“어떤 계좌가 구조적으로 수상한가?”
(c) 교통	도로망 + 시계열	공간 × 시간 반복 갱신	“앞으로 N분 후의 교통량은?”

재료가 그대로 실전에 옮겨진다

- 세 케이스 모두, 이 강의 “재료”인 **그래프 자체**가 **도메인 지식에서 직접 만들어지는 형태** (접촉 그래프, 거래 그래프, 도로망).
- 이 강의 “연산”인 **반복해서 이웃과 값을 합친다**는 것이 각 도구의 **핵심**에 들어 있다.

그래프 학습에서 “모델링”이 “프로그래밍”보다 중요한 이유

도메인이 달라질수록 그래프를 어떻게 만드는가 (어떤 것을 노드로, 어떤 것을 엣지로 하는가)의 선택이 **모델의 성능을 결정한다**.

다음으로 가는 길: ML2 강화학습으로

- (c)의 “시계열을 그래프 위에 얹는다”는 생각은 **ML2의 강화학습**으로 자연스럽게 이어진다.
- **MDP**는 “상태를 노드로, 행동을 전환으로 놓은 그래프”와 같은 구조를 갖고.
- Q-함수 / 가치함수는 그 위에서 “반복해서 값을 갱신한다”는 연산을 수행 (ML2 Chapter 4의 **벨만 방정식**).

13.4 → ML2

13장의 “그래프 위에서 값을 갱신하는 감각”이, ML2의 “상태-행동 그래프 위에서 가치를 갱신하는 감각”으로 이어진다.

이번 주차 정리

- ① **13.1 무작위 걷기 = 마르코프 체인** — 전이확률 $P(i \rightarrow j) = 1/\deg(i)$, 무자기성, 정상분포 π^* 가 **도에 비례**. 걷기 말뚝치 $\rightarrow$ skip-gram $\rightarrow$ 카라테 두 파벌이 공간적으로 갈라짐.
- ② **13.2 Node2Vec** — 걷기 경로 = 문장, 노드 = 단어로 skip-gram을 그대로. p, q = BFS형(역할) vs DFS형(커뮤니티) 다이얼. 실수: **무방향 대칭화 누락**.
- ③ **13.3 PageRank** — **같은 걷기에 다른 질문** (머무는 확률 = 중요도). 댐핑 + teleport, 거듭제곱법 (바나흐 고정점으로 수렴), 4-노드 손계산. $\neq$ 도.
- ④ **13.4 확장** — 단백질(AlphaFold $\rightarrow$ DeepFRI) · 금융(AML World) · 교통(STGCN). “반복해서 이웃과 값을 합친다”가 실전 도구의 핵심.

다음 주 — Chapter 14. 표현학습

“함께 자주 나타나는 것은 가까운 벡터로 모인다”의 **텍스트 버전** — **PCA**와 **word2vec**. 이번 장이 그래프 버전을 먼저 보여줬고, 14장에서 그 원형을 본문에서 배운다. (“반복해서 값을 갱신한다”는 감각은 ML2의 벨만 방정식으로 이어진다.)